VACF Running-Diffusion Plotting Specification

VP2: Green-Kubo D(t) Visualization

mdstats

2026-07-15

Contents

Purpose

This document specifies

`mdstats/plotting/vacf_transport.py`

and its public function

```
plot_vacf_diffusion(result, ...)
```

The function visualizes one or more already computed `VACFDiffusionResult` objects. It does not recompute a VACF, perform numerical quadrature, fit a long-time tail, choose a plateau, or declare a converged transport coefficient.

VP2 is implemented in `mdstats 0.19.9a0`.

Physical background

For an isotropic system in d spatial dimensions, the running Green-Kubo self-diffusion integral is

$$D(t) = \frac{1}{d} \int_0^t \langle \mathbf{v}_i(0) \cdot \mathbf{v}_i(\tau) \rangle \, d\tau.$$

For one Cartesian direction α,

$$D_\alpha(t) = \int_0^t \langle v_{i\alpha}(0) v_{i\alpha}(\tau) \rangle \, d\tau.$$

The time-correlation transport framework follows Green [1] and Kubo [2]. The input result and cumulative trapezoidal quadrature are owned by `integrate_vacf_to_diffusion()` and its analysis specification. VP2 only renders the stored finite-time running integral.

The limit

$$D = \lim_{t \to \infty} D(t)$$

is meaningful only when a stable long-time regime exists and the trajectory is sufficiently sampled. Oscillation, return toward zero, or negative finite-time values can be physically meaningful for trapped or vibrational motion. The plotter therefore never labels the final sample as an asymptotic diffusion coefficient.

Provenance

Borrowed software machinery

Rendering uses Matplotlib, cited to Hunter [3].

Reused mdstats scientific machinery

The plotter consumes only `VACFDiffusionResult`, whose numerical and physical contract is defined by the Green-Kubo transport module. Unit conversion from $\mathrm{\AA}^2/\mathrm{ps}$ to cm^2/s is already exposed by that result type.

mdstats-specific design

The following are mdstats decisions:

- one plotting function for a single result, sequence, or label mapping;
- explicit time and diffusion units;
- no automatic plateau selection or tail fitting;
- an optional zero reference line;
- deterministic default labels for scalar and Cartesian curves;
- returning `(Figure, Axes)` without calling `show()` or saving files.

Public API

```python
from collections.abc import Mapping, Sequence
from typing import Literal

from matplotlib.axes import Axes
from matplotlib.figure import Figure


def plot_vacf_diffusion(
    result: (
        VACFDiffusionResult
        | Sequence[VACFDiffusionResult]
        | Mapping[str, VACFDiffusionResult]
    ),
    *,
    ax: Axes | None = None,
    time_unit: Literal["fs", "ps", "ns"] = "ps",
    diffusion_unit: Literal["angstrom2/ps", "cm2/s"] = "cm2/s",
    label: str | None = None,
    labels: Sequence[str] | None = None,
    title: str | None = None,
    show_zero_line: bool = True,
    grid: bool = True,
) -> tuple[Figure, Axes]:
    ...
```

Input contract

Every plotted object must be a validated `VACFDiffusionResult` containing:

```
lag_times
running_diffusion_a2_per_ps
running_diffusion_cm2_per_s
component
dimensions
weighting
integration
```

The plotter does not accept arbitrary arrays because the result type carries the physical weighting and unit guarantees needed for a correct label.

Multiple-result contract

Accepted forms are:

```
plot_vacf_diffusion(na_result)
plot_vacf_diffusion([na_result, k_result], labels=["Na", "K"])
plot_vacf_diffusion({"Na": na_result, "K": k_result})
```

For one result, `label=` may be supplied. For a sequence, `labels=` must have the same length. For a mapping, its keys are authoritative labels and `labels=` is invalid.

Unit contract

Time

The stored time axis is in ps. Display conversion is

$$t_{\mathrm{fs}} = 10^3 t_{\mathrm{ps}}, \qquad t_{\mathrm{ns}} = 10^{-3} t_{\mathrm{ps}}.$$

Diffusion

The canonical stored running integral is in $\mathrm{\AA}^2/\mathrm{ps}$. The displayed SI-derived unit uses

$$ 1\ \mathrm{\AA}^2/\mathrm{ps}

10^{-4}\ \mathrm{cm}^2/\mathrm{s}. $$

Unit changes affect plotted copies only and never mutate the result.

Rendering contract

The function:

1. resolves and validates labels;
2. converts the time coordinate for display;
3. selects the requested diffusion unit;
4. draws one line per result;
5. optionally draws a light $D = 0$ reference;
6. sets labels, title, grid, and legend when needed;
7. returns the owning `Figure` and `Axes`.

It does not smooth, interpolate, truncate, normalize, or resample the curve.

Example workflow

The package example

```
examples/vacf_dynamics/vasp_contcar_trajectory_vdos_diffusion.py
```

reads a watcher-generated VASP `TRAJECTORY`, computes a mass-weighted VACF-derived VDOS, computes a uniformly weighted self VACF for a selected mobile species, integrates it into $D(t)$, and saves separate VDOS and running self-diffusion figures.

For the supplied Na-LTA file:

```
python examples/vacf_dynamics/vasp_contcar_trajectory_vdos_diffusion.py \
    TRAJECTORY \
    --timestep-fs 1.0 \
    --diffusion-species Na \
    --output-dir vacf_outputs
```

Error handling

The function rejects:

- unsupported result types;
- empty result collections;
- non-string or empty labels;
- inconsistent sequence-label lengths;
- unsupported time or diffusion units;
- non-boolean display switches;
- invalid axes objects.

Tests

Focused tests verify:

1. top-level and plotting-package exports;
2. exact use of stored $\mathrm{\AA}^2/\mathrm{ps}$ values;
3. exact conversion to cm^2/s;
4. fs, ps, and ns horizontal axes;
5. mapping and sequence labels;
6. optional zero reference;
7. no mutation of result arrays;
8. invalid-option failures;
9. real-file generation of separate VDOS and $D(t)$ figures.

References

[1] M. S. Green, “Markoff random processes and the statistical mechanics of time-dependent phenomena. II. Irreversible processes in fluids,” *Journal of Chemical Physics* **22**, 398-413 (1954). DOI: `10.1063/1.1740082`.

[2] R. Kubo, “Statistical-mechanical theory of irreversible processes. I. General theory and simple applications to magnetic and conduction problems,” *Journal of the Physical Society of Japan* **12**, 570-586 (1957). DOI: `10.1143/JPSJ.12.570`.

[3] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering* **9**, 90-95 (2007). DOI: `10.1109/MCSE.2007.55`.